Champion Creation — Volume 2 Addendum

For: anyone, human or AI, building a champion with *02 — Champion Creation: Technical and Experimental Guide* (Volume 2). **Applies to:** branch `claude/vesper-scifi-variety`, native **0.10.46**, champion pack **0.2.47**. **How it was made:** every page of Volume 2 was checked against the repository. Each fact here is cited as `path:line` at this version, and the citations are checked by a script.

Volume 2 holds up well. Its SDK surface, file paths, field types, categories and tags, the ABI 9 rule, the version test, `perf.flag`, the Manager commands and the example kit all check out against the code. This addendum covers three things: what has changed since Volume 2's snapshot, the few places it is wrong, and six lessons it is missing. Those six lessons are the ones that cost the most rounds to learn.

1. How to read Volume 2 today

- **Its snapshot is `73d154f`** (the PR #30 merge, native 0.10.43). The source register's line numbers (R001–R185) resolve only there. To read a cited line as Volume 2 saw it, run `git show 73d154f:<path> | sed -n '<line>p'`. At HEAD, search for the symbol instead.
- **Never copy version numbers from a guide.** Read them from `native/tfm2_custom_ai/src/lib.rs:265` (`VERSION`) and bump all four places together (§8).
- **Numbers that have moved since the snapshot:**
  - the rank fixtures run 24 seeds, not 12 (`native/tfm2_custom_ai/src/vesper_tests.rs:1120`);
  - the native test count is 258, not 254;
  - the parity check has 674 simulation rows, not 523;
  - the Vesper verifier covers 984 views over 66 sheets.

2. Errata

Volume 2	What it says	What is true	Where
p. 1 and headers	native 0.10.43, pack 0.2.43	0.10.46 / 0.2.47 at this addendum	`native/tfm2_custom_ai/src/lib.rs:265`
pp. 39–40, 61	the rank fixture is 12 seeds; Worldbreaker wins 12/12	24 seeds: with 12, a handful of fights flipping on a single pull moved the win count by two. Worldbreaker 16/24, Spark 0/24	`native/tfm2_custom_ai/src/vesper_tests.rs:1120`
p. 47	integrator-owned files: `lib.rs` and the pack `mod.mod_info`	also `native/tfm2_custom_ai/mod.mod_info` and `mods/tfm2_custom_ai/mod.mod_info`; all three native versions must agree	`native/tfm2_custom_ai/src/lib.rs:1826`

Volume 2	What it says	What is true	Where
p. 14	the template uses `declare_stable_mod! (init, requires = 9)`	correct for a new mod. The repo itself declares `declare_stable_mod! (init);`, which defaults to the baseline ABI 1, while it calls `add_native_passive` (an ABI 9 feature). It works because today's host is ABI 9; a new mod should state `requires = 9`	`native/tfm2_custom_ai/src/lib.rs:1793`, `native/mod-api-stable/src/entry.rs:39`, `native/mod-api-stable/src/lib.rs:175`
p. 16	avoid encoding state in buff names	right for a resource your own rules depend on. But it is **the** way to tell the game's AI about zones (§6), and the repo relies on it	`native/tfm2_custom_ai/src/valorant.rs:207`
p. 17	Vesper's placeholder action shell is "not a design pattern to copy"	for a champion whose rules are all native, the `never()` shell **is** the pattern: the built-in planner must never cast those slots itself	`Claude outputs/coder/coder_data.py:24`
p. 24	Manager flags: `--build`, `--update`, `--check`, `--logs`	also `--perf` (performance log on/off) and `--editor`	`tools/manager/src/main.rs:522`
p. 42	Rewind's 180-tick punch against 120 ticks of history; Plating's text vs code	both were real bugs, now fixed (§10)	`native/tfm2_custom_ai/src/vesper_data.rs:48`

3. Facing: what the game mirrors and what it never does

The rule. When a champion walks left, the game mirrors **only its own body sprite**, `champions/<id>#sheet.png` (`Sprite kit/README.txt:30`). It never mirrors:

- **buff overlays** (`view_buffs`). Our notes said so three times (Scribble's skins, Levi's wings, the Coder's outfits), always as "keep it symmetric";
- **view effects** drawn by native code. Directional art ships `_a{heading}` variants instead.

Why it matters. Vesper and the Unified Theory draw their visible body as a buff overlay that covers the base sprite. For months they never turned around. Volume 2's p. 21 ("faces right, mirrored by the game") is true only for the base sprite.

The fix:

1. **Art.** For every asymmetric body layer, ship a mirrored `<tag>_l` copy flipped around the same anchor (the frame centre), for example `Image.transpose(FLIP_LEFT_RIGHT)`. See `Claude outputs/vesper/art.py:215` and `Claude outputs/unified_theory/mastery_art.py:255`. Put the copies in the same sheet or in a `_l` sheet. If the sheet is palette-quantised, quantise both copies together, otherwise the mirror is not exact.

2. **Native.** Track facing and append `_l` to the body layer's buff name. The shared helper is `native/tfm2_custom_ai/src/facing.rs:16`. It uses DIO's rule, the one checked in game (`native/tfm2_custom_ai/src/batch2.rs:662`):
   - face the nearest enemy within 90000;
   - otherwise face the way it walks, past a 200-unit dead zone;
   - standing still keeps the facing;
   - a 6-tick hold stops flicker.

   Examples: `native/tfm2_custom_ai/src/vesper.rs:171` and `native/tfm2_custom_ai/src/unified_theory.rs:398`.
3. **Don't mirror** rank badges, the HUD, or anything with text (the scientists' "E=mc²" would read backwards).
4. **Verify.** Assert that every `_l` frame equals its right-facing frame flipped (`tools/verify_vesper.py:101`). Also review both facings in the editor.

The kit's `native-example/src/facing.rs` is a standalone copy with tests.

4. Performance, with numbers

Two causes have produced every lag report so far:

- the **number of effects started** per second;
- the **size of the sheets** the game must decode when an effect first appears.

Our own code's CPU time has never been the cause (`perf.flag` measures it: `native/tfm2_custom_ai/src/perf.rs:2`).

Effect budget. Use a token bucket, not a per-tick cap alone. In `native/tfm2_custom_ai/src/vesper_core.rs:530` (`fx`) and `:541` (`fx_low`):

- 12 tokens, one back every 2 ticks, at most 6 per tick (`native/tfm2_custom_ai/src/vesper_data.rs:30`);
- opens, closes, hits and casts may spend any token;
- ambient re-spawned visuals (looping layers, projectiles in flight) only spend above a reserve of 4;
- a refused layer retries the next tick instead of waiting a whole 48-tick loop (`native/tfm2_custom_ai/src/vesper_core.rs:2800`).

The fixture asserts the busiest second never exceeds one bucket plus a second of refill.

Sheet hygiene:

- Split look-, rank- and tier-dependent art into **one sheet per look or tier** (`Claude outputs/vesper/rift_art.py:1757`). A match only decodes its own Vesper's art: about 8 MiB of rifts instead of 80.
- Isliid's spawn hitches were traced to giant shared sheets decoded when an effect first appeared (`docs/playtest-notes.md:2391`).
- Keep every atlas at 2048 px or less.

View definitions. Keep the count in the low thousands. Isliid's 9,514 view effects (a 2.3 MB data file) were the lead lag suspect and were cut to 5,034 with no visible change (`docs/playtest-notes.md:2326`, `:2342`).

Memory. Budget the decoded RGBA, width × height × 4 summed over every sheet (`tools/verify_vesper.py:99`: Vesper is at 189.4 of 190 MiB). Pay for new art by packing better:

- one symmetric crop per animation;
- shelf-pack the tallest animations first (`Claude outputs/vesper/art.py:41`);
- shorter canvases where the art allows.

Native:

- never deep-clone large state every tick; take the core out of its `Option`, step it, and put it back;
- gather per-think caches once (the live rifts and chain), not once per candidate;
- build log strings only while the trace has room.

Measure it in game. Turn on Manager menu 7 (`--perf`) and play one match. `perf_log.txt` shows our milliseconds per hook, the effects per second and the **longest gap between ticks**. A long gap with few milliseconds in our code means the game itself (drawing and effects) is the bottleneck.

5. The atlas contract

- **Anchor.** `.fanim` frames have only `x, y, w, h, duration`; there is **no origin field**. The game puts the **frame centre** on the champion's position, with feet 20 px below the centre on the 48×56 template. Crop every animation symmetrically around the centre with one crop shared by all its frames, otherwise it wobbles.
- **Frames.** 8 frames per animation, and 16 for animated rank emblems (`tools/verify_vesper.py:92`).
- **Lifetime.** A view effect lives for the sum of its frame durations. The `ticks` argument does not stretch it, so match the art to the native lifetime.
- **Scale.** 1 px is 950 world units, and 60 ticks is one second.

6. Talking to the game's AI

There is no engine API for "this area is dangerous". The working pattern has three parts.

1. **Publish** each zone as a **buff name on the owner**:
   - Omen's smokes are `oms:x:y:end` (`native/tfm2_custom_ai/src/valorant.rs:207`), Steve's walls `sbw:…`, Vesper's rifts and Kidnap fields `vsr:x:y:r:until` / `vsk:…` (`native/tfm2_custom_ai/src/vesper.rs:54`, `:197`).
   - Add the buff when the zone opens and remove it when it closes; never re-add it every tick.
   - Round positions so a moving zone only republishes when it really moved. Buff names are capped at 64 bytes.
   - Call `steve::note_map_buff(seed)` (`native/tfm2_custom_ai/src/steve.rs:509`). Without it the input AI's move rules stay idle and never read your buffs (`native/tfm2_custom_ai/src/steve.rs:522`).
2. **Steer** in the shared input AI, `steve::WallAi` (`native/tfm2_custom_ai/src/lib.rs:1789`). It is one AI for every player.
   - Rewrite **move orders only** (`native/tfm2_custom_ai/src/steve.rs:910`).
   - **Never inject, hold or retarget casts or attacks.** Doing so desynchronised the game's planner, which then panicked on every parallel sim thread (`docs/playtest-notes.md:134`).
   - Vesper's hook is at `native/tfm2_custom_ai/src/steve.rs:974`, and her rule is at `native/tfm2_custom_ai/src/vesper.rs:93`:
     - her enemies never walk **into** a zone;
     - they walk **out of an armed field** (the Kidnap ring), which is their dodge;
     - a rift that opens **under** them is fought in, not fled.

Walking out of every fresh rift made every rank's fights much harder for Vesper, so that variant was rejected.

3. **Other mod AIs** read the same zones through `batch2::danger_zones` (`native/tfm2_custom_ai/src/batch2.rs:134`); Minato's teleports and dodges use it.

Mirror the rule in your test arena, so the fixtures measure enemies that avoid your zones (`native/tfm2_custom_ai/src/vesper_tests.rs:809`). The kit's `native-example/src/zones.rs` is a standalone version with tests.

7. Native-driven champions

When every rule lives in native code (the Coder, Vesper, the Unified Theory):

- **Data shell.** `skill`, `skill2` and `ult` are `never()` actions: a cooldown of 5184000, `EnemyChampionInCC`, range 1, an empty `Combine` (`Claude outputs/coder/coder_data.py:24`). The built-in planner then never casts them, and the native passive owns the kit.
- **Attach** the passive with `"passive": {"passive_ref": "tfm2_custom_ai:<name>", "params": {}}`.
- **Register** it in `init` (`native/tfm2_custom_ai/src/lib.rs:1787`), and add the champion to `MOD_POWER` (`native/tfm2_custom_ai/src/lib.rs:82`).
- **Split** the code into a deterministic core (snapshot in, commands out, no host handles) and a thin adapter that applies the commands. Port the core to JS for the editor and keep checked-in parity vectors (`tools/check_vesper_parity.js`). Include effects and every gameplay stat in the vectors, not just damage.

8. Release hygiene

- **Four version fields, always together:**
  1. `native/tfm2_custom_ai/src/lib.rs:265`;
  2. `native/tfm2_custom_ai/mod.mod_info`;
  3. `mods/tfm2_custom_ai/mod.mod_info`;
  4. the pack's requirement `>=` in `mods/tfm2_custom/mod.mod_info`.

  In round 108 a disagreement made the game disable **every** champion (`native/tfm2_custom_ai/src/lib.rs:1826`).
- **Categories and tags** must come from the accepted lists (`tools/manager/src/core.rs:68`), or the game skips the champion. The Manager's Check reports it (`tools/manager/src/core.rs:320`).
- **Renamed or split sheets** leave the old files in players' game folders. List them in `STALE_FILES` (`tools/manager/src/core.rs:12`) and rebuild `TFM2 Mod Manager.exe`; the Manager only deletes what it knows about.
- **Ship what you tested.** Record the DLL's SHA-256 in the verification report and compare it with the build output.
- **Don't run `--update --yes` on a working branch.** It pulls and may switch branches.

9. Fixtures that don't lie

- **Seeds.** At 12 seeds, the new Kidnap seemed to cost Worldbreaker four wins. Over 36 seeds it cost one (25 vs 26), plus about 6% damage. Use at least 24 seeds and widen with `VESPER_SEEDS` when tuning. Scale margins with the sample (`native/tfm2_custom_ai/src/vesper_tests.rs:1120`).

- **Relative assertions.** A Spark's random swipes hit about one in two, and luck moved that across a fixed 50% bar. "A Worldbreaker hits two or more a swipe and three times a Spark's rate" is the claim that matters.
- **Variety.** Assert that no technique takes more than a share of plays (`native/tfm2_custom_ai/src/vesper_tests.rs:1161`). A planner that always picks its best play becomes repetitive.
- **Your own fixture should be able to fail.** Before trusting a new assertion, reintroduce the old bug in a scratch copy and watch it fail (Volume 2, pp. 37–38).

10. Live bugs that Volume 2's tests found, fixed in 0.10.46

- **Rewind.**
  - Bug: its full-strength punch looked 180 ticks back, but the core kept 120 ticks (21 samples), so it silently did nothing.
  - Fix: the history is now 180 ticks (`native/tfm2_custom_ai/src/vesper_data.rs:48`). A teleport now only zeroes velocity (`native/tfm2_custom_ai/src/vesper_core.rs:1519`), so Rewind can still send a target back to before its blink.
  - Test: `native/tfm2_custom_ai/src/vesper_tests.rs:671`.
- **Plating.**
  - Bug: the text said "nearest ally", but the code shields the lowest-health ally near the rift (`native/tfm2_custom_ai/src/vesper_core.rs:1036`). The behaviour is the sensible one.
  - Fix: the text now says what it does.

11. Additions to the gates (p. 55) and the checklist (p. 60)

Paste these under Volume 2's checklist:

- ☐ All four version fields agree; `every_version_agrees` passes.
- ☐ `category` and `tags` come from the accepted lists; Manager Check reports no data errors.
- ☐ Body art faces right. Every overlay body layer has an exact `_l` mirror, the native code picks it, and both facings were reviewed.
- ☐ Badges, HUD and text are not mirrored.
- ☐ Effects use a budget with a low-priority reserve; the fixture asserts the peak per second.
- ☐ Look-dependent art is split per look or tier; every atlas is 2048 px or less; 8 frames (16 for emblems); one symmetric crop per animation.
- ☐ The decoded RGBA total is measured and within the champion's budget; the view-definition count is in the low thousands.
- ☐ No per-tick deep clones; per-think caches are built once.
- ☐ Danger zones are published as buff names, added on open and removed on close, with `note_map_buff`.
- ☐ Input-AI rules rewrite moves only, never casts or attacks.
- ☐ Native-driven champions use `never()` actions, a `passive_ref`, registration in `init` and `MOD_POWER`, and `requires = 9` for new mods.
- ☐ Renamed or split sheets are listed in `STALE_FILES`, and the Manager exe is rebuilt.
- ☐ Fixtures use 24 or more seeds, relative assertions and a variety cap; JS/native parity vectors cover effects and every stat.
- ☐ The shipped DLL's SHA-256 is recorded; "automated checks passed, live acceptance pending" is stated until a real match is played.

The example kit

`champion-authoring-examples/` is in the repository, not installed and not shipped (the Manager only copies `mods/*`).

- **What it contains:** your data prototype and native example unchanged, plus `facing.rs`, `zones.rs` and `budget.rs`.
- **Tests:** 8 in total. Run `cargo test --manifest-path champion-authoring-examples/native-example/Cargo.toml`.